

Linux Netstat Utility using Socket Programming

(Client-Server Implementation)

Design Documents -Version 0.0

Document Control

Revision History

Date	Version	Author	Description	
18-05-2026	0.0		Initial HLD Document	

Team Members

Naveen Kumar	
Vineetha	
Keerthana	
Amose	

Table of Contents

1) HIGH-LEVEL DESIGN

1. INTRODUCTION	6
1.1 PURPOSE	6
1.2 SCOPE	6
1.3 DEFINITIONS	6
1.4 OVERVIEW	7
2.GENERAL DESCRIPTION	8
2.1 PRODUCT PERSPECTIVE	8
2.2 TOOLS USED	8
2.3 GENERAL CONSTRAINTS	8
2.4 ASSUMPTIONS	9
3.DESIGN DETAILS	9
3.1 MAIN DESIGN FEATURES	9
3.2 SYSTEM ARCHITECTURE	10
3.3 DATA FLOW DIAGRAM	11
3.4 FILES	16
3.5 USER INTERFACE	16
3.6 ERROR HANDLING	16
3.7 HELP	17
3.8 PERFORMANCE	17
3.9 SECURITY	17
3.10 RELIABILITY	18
3.11 MAINTAINABILITY	18
3.12 PORTABILITY	18
3.13 REUSABILITY	18
3.14 APPLICATION COMPATIBILITY	18
3.15 RESOURCE UTILIZATION	19
3.16 MAJOR CLASSES	19

2) LOW-LEVEL DESIGN	1
4. INTRODUCTION	1
4.1 PURPOSE	1
4.2 DOCUMENT CONVENTIONS	1
4.3 INTENDED AUDIENCE AND READING SUGGESTION	2
4.4 REFERENCES	2
5. DETAILED SYSTEM DESIGN	2
5.1 DESIGN DESCRIPTION	2
5.2 CONNECTION ESTABLISHMENT BETWEEN CLIENT AND SERVER	3
5.3 USE CASE DIAGRAM	4
5.4 DESIGN AND IMPLEMENTATION CONSTRAINTS	4
5.5 USER INTERFACE	5
5.6 SECURITY	5
5.7 SERVER MAIN LOOP FLOWCHART	5
5.8 CLIENT REQUEST FLOWCHART	6
5.9 STATE DIAGRAM	7

High Level Design

1. Introduction

1.1 Purpose

The purpose of this High-Level Design (HLD) document is to describe the overall structure and working of the Netstat Utility system before implementation. This document explains how different modules of the project interact with each other and how the complete system behaves.

The project is developed using Client-Server Architecture in which the client sends requests to the server using TCP socket communication. The server processes those requests by executing Linux Netstat commands and sends the output back to the client.

This document helps developers testers, and reviewers understand the complete workflow of the system.

1.2 Scope

The scope of this project is to provide a Linux-based utility that remotely displays network-related information using client-server communication.

The system allows users to:

- View protocol statistics
- View routing table information
- View active TCP connections
- View active UDP connections
- View LISTEN and ESTABLISHED connections
- View browser-related network connections

The server processes all requests and supports multiple clients simultaneously using multithreading.

1.3 Definitions

Netstat

Netstat is a Linux networking utility used to display network statistics, routing tables, and active network connections.

Client

A client is any computer hardware or software device that requests access to a service provided by a server. Clients are typically seen as the requesting program or user in a client-server architecture.

Server

A server is a computer program or device that provides a service to another computer program and its user, also known as the client. In a data center, the physical computer that a server program runs on is also frequently referred to as a server.

TCP

Transmission Control Protocol (TCP) is a reliable communication protocol used between client and server.

Concurrent Server

A concurrent server is capable of handling multiple clients simultaneously.

1.4 Overview

This HLD document contains the following sections:

- Introduction

A brief explanation about the purpose, aim, scope and design format of the proposed system.

- General Description

This section is all about the general constraints, assumptions and design aspects associated with the proposed system. The product perspective will give an overall description of the system.

- Design Details

This section documents the detailed design of all modules associated with the development of the proposed system.

- Architecture
- System Flow
- Error handling
- Security and Reliability

2. General Description

2.1 Product Perspective

The Netstat Utility system follows a Client-Server Architecture.

Working Flow:

1. Client connects to server
2. User selects an option
3. Client sends request
4. Server executes Netstat command
5. Server sends output back to client
6. Client displays result

The server supports multiple clients simultaneously.

2.2 Tools Used

- Linux Operating System
- C/C++ Programming Language
- GCC/G++ Compiler
- TCP Socket Programming
- Netstat Utility
- pthread Library

2.3 General Constraints

- The application should run in Linux environment.
- TCP protocol must be used.
- Netstat command should be available in the system.
- Network connection should be stable.

2.4 Assumptions

- Server should always be running.
- Client and server systems should be connected through network.
- Linux terminal is required.

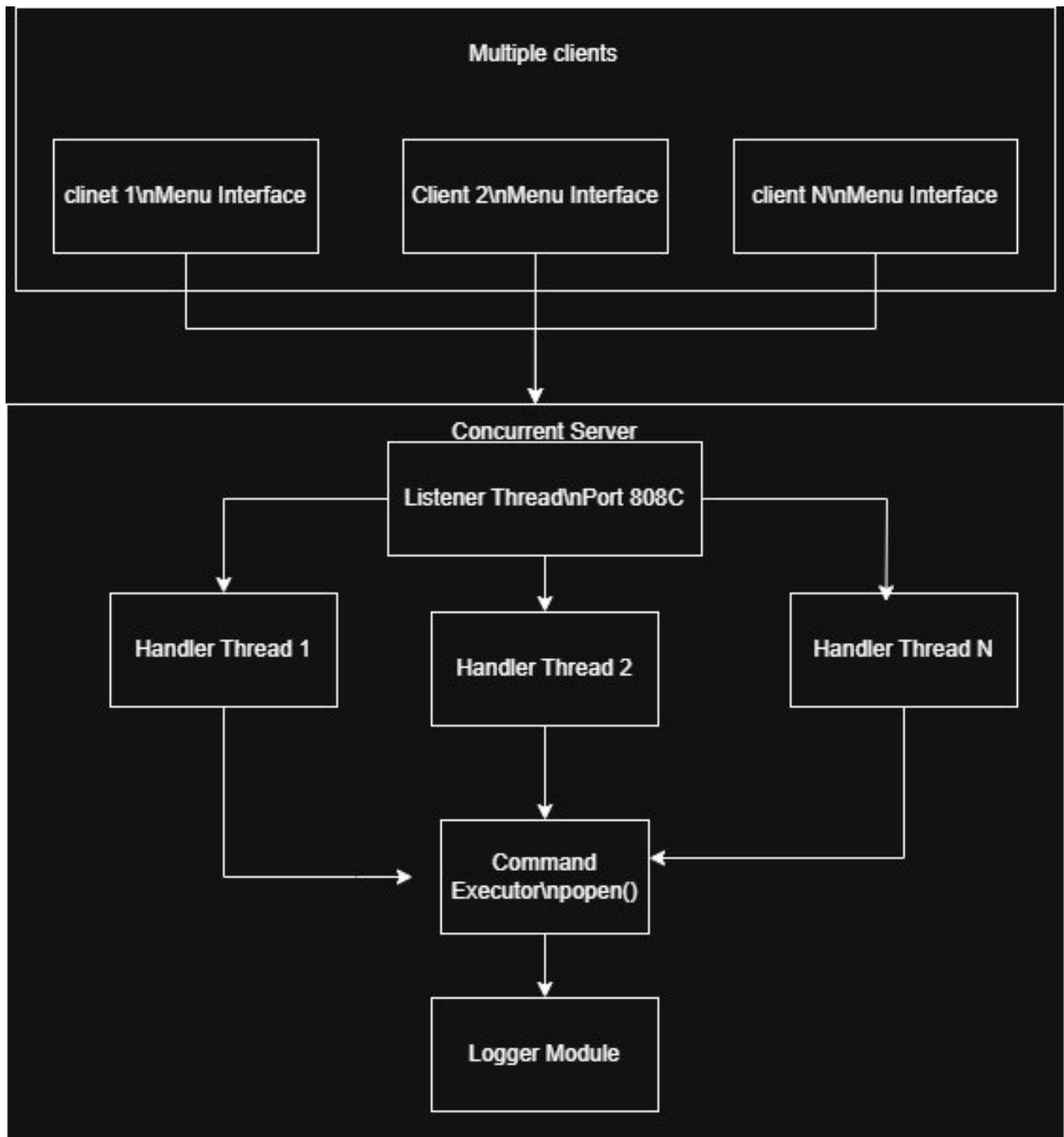
3. Design Details

3.1 Main Design Features

Main features of the project include:

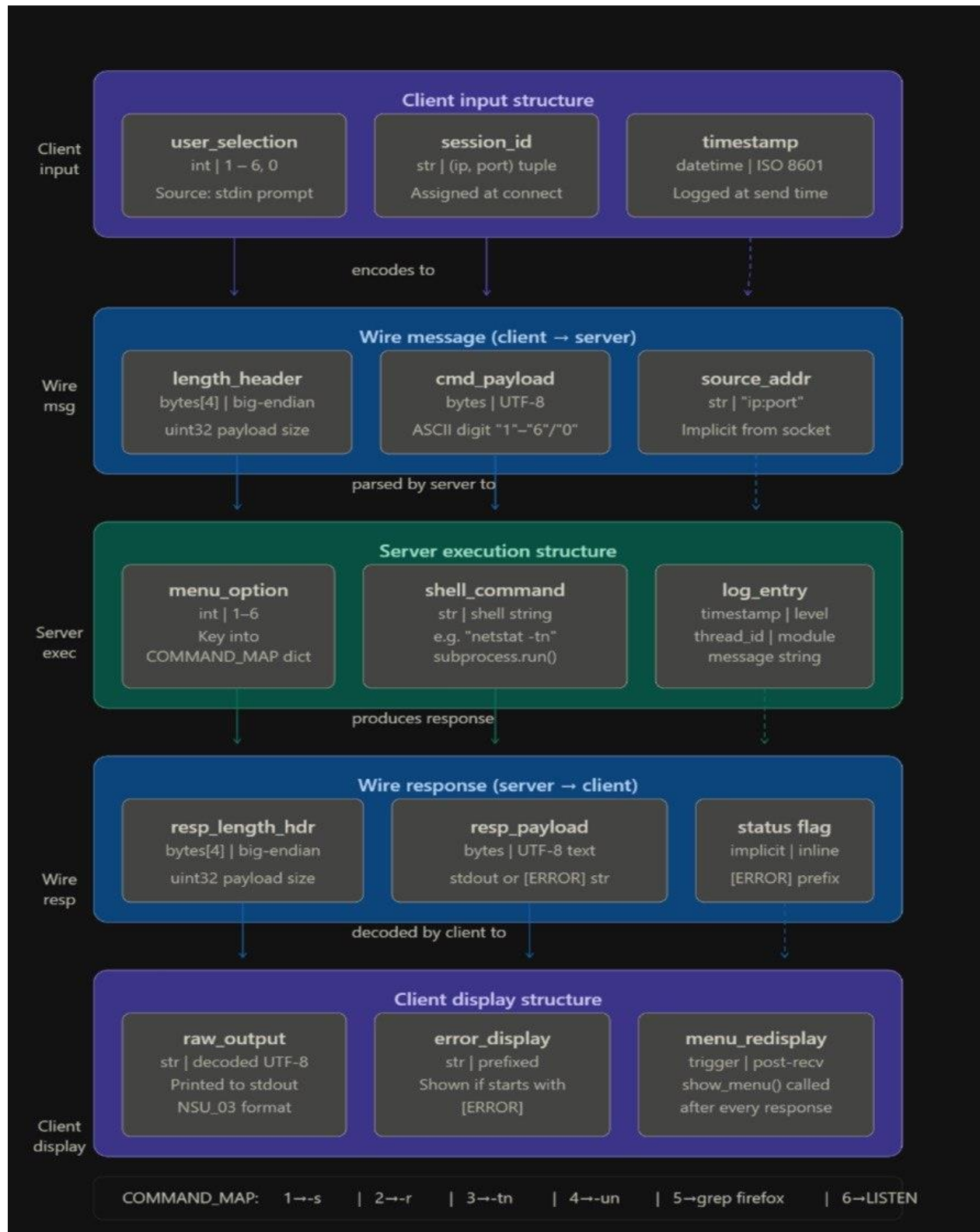
- Client-server communication
- Menu-driven interface
- Concurrent server handling
- Linux Netstat command execution
- Logging mechanism
- Error handling

3.2 System Architecture

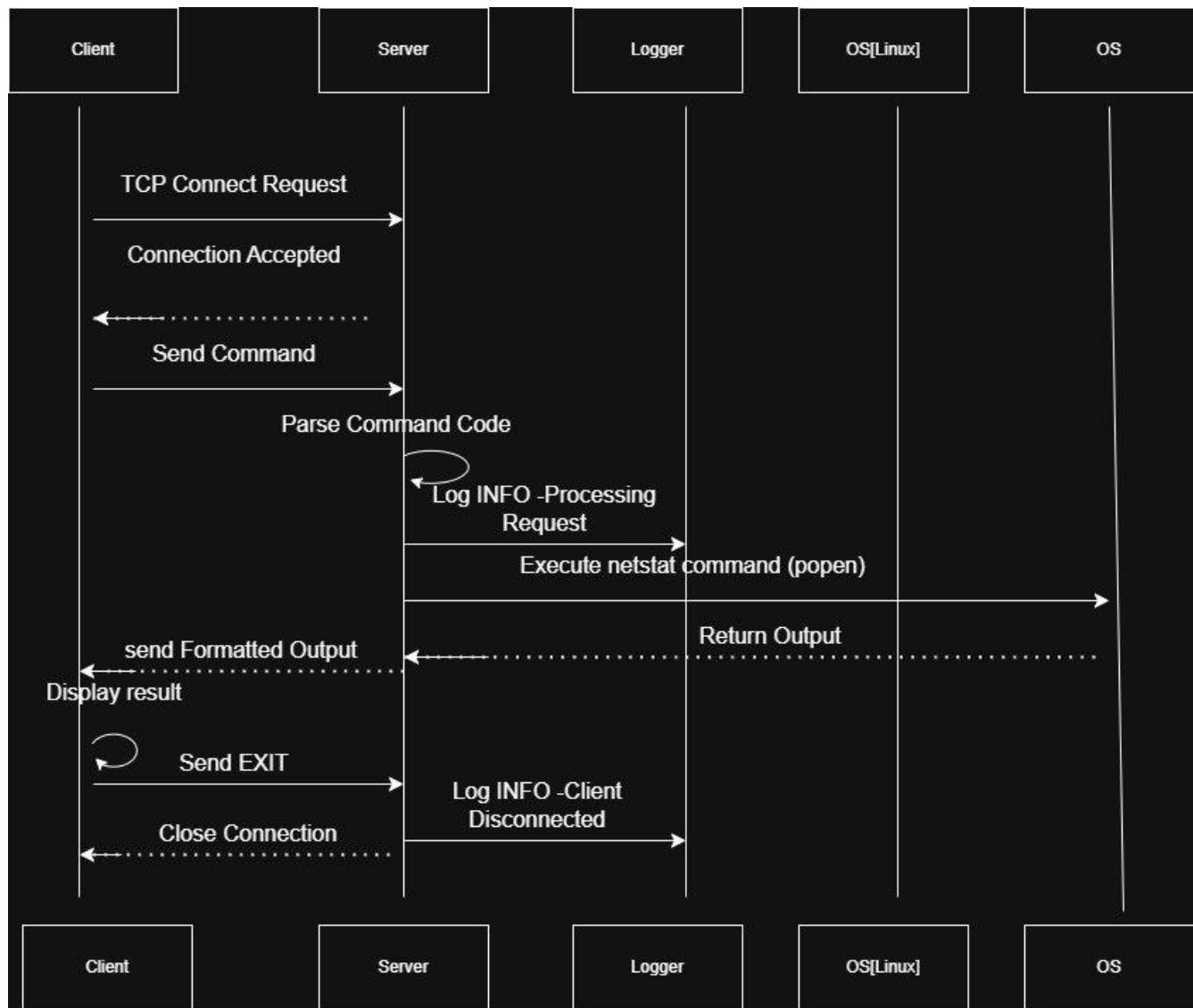


3.3. Data Flow Diagram

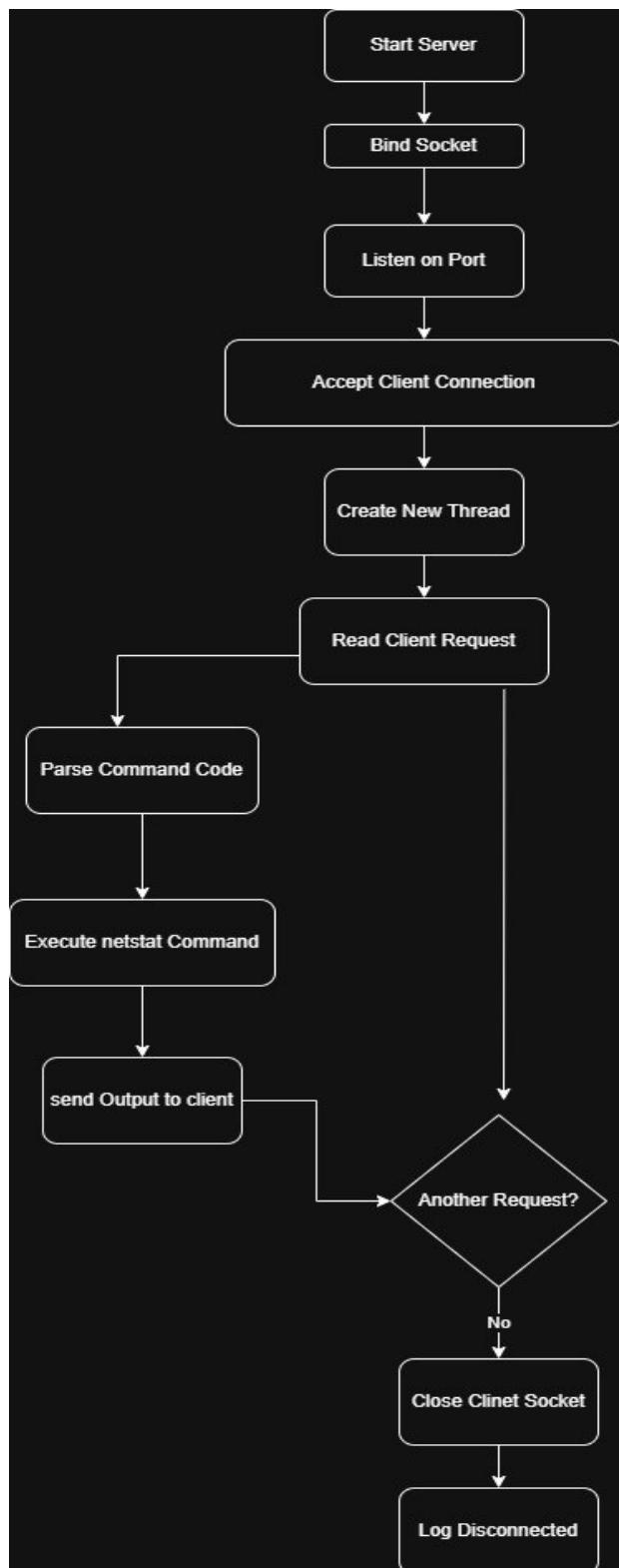
3.3.1 Data Mapping Information



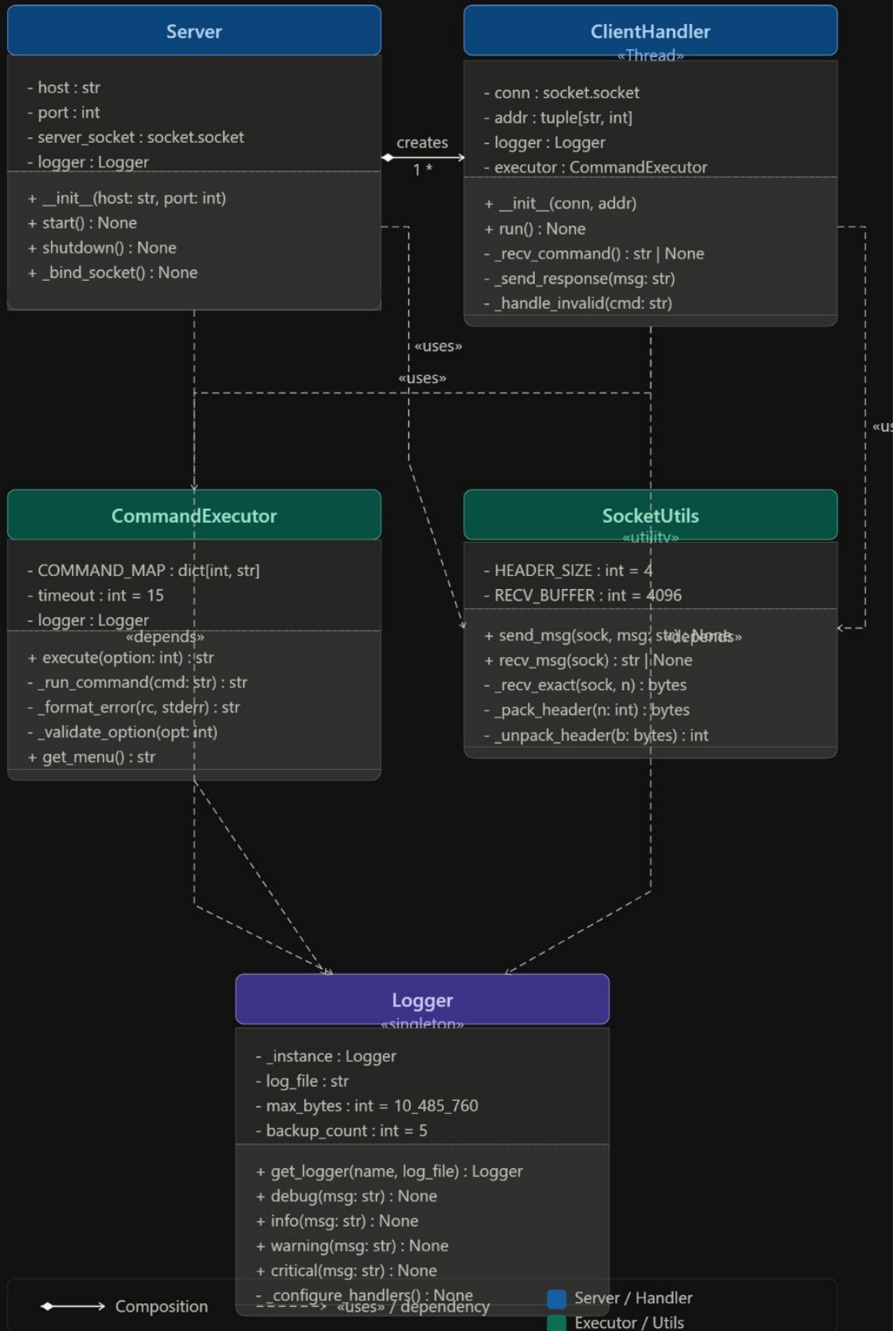
3.3.2 Sequence Diagram



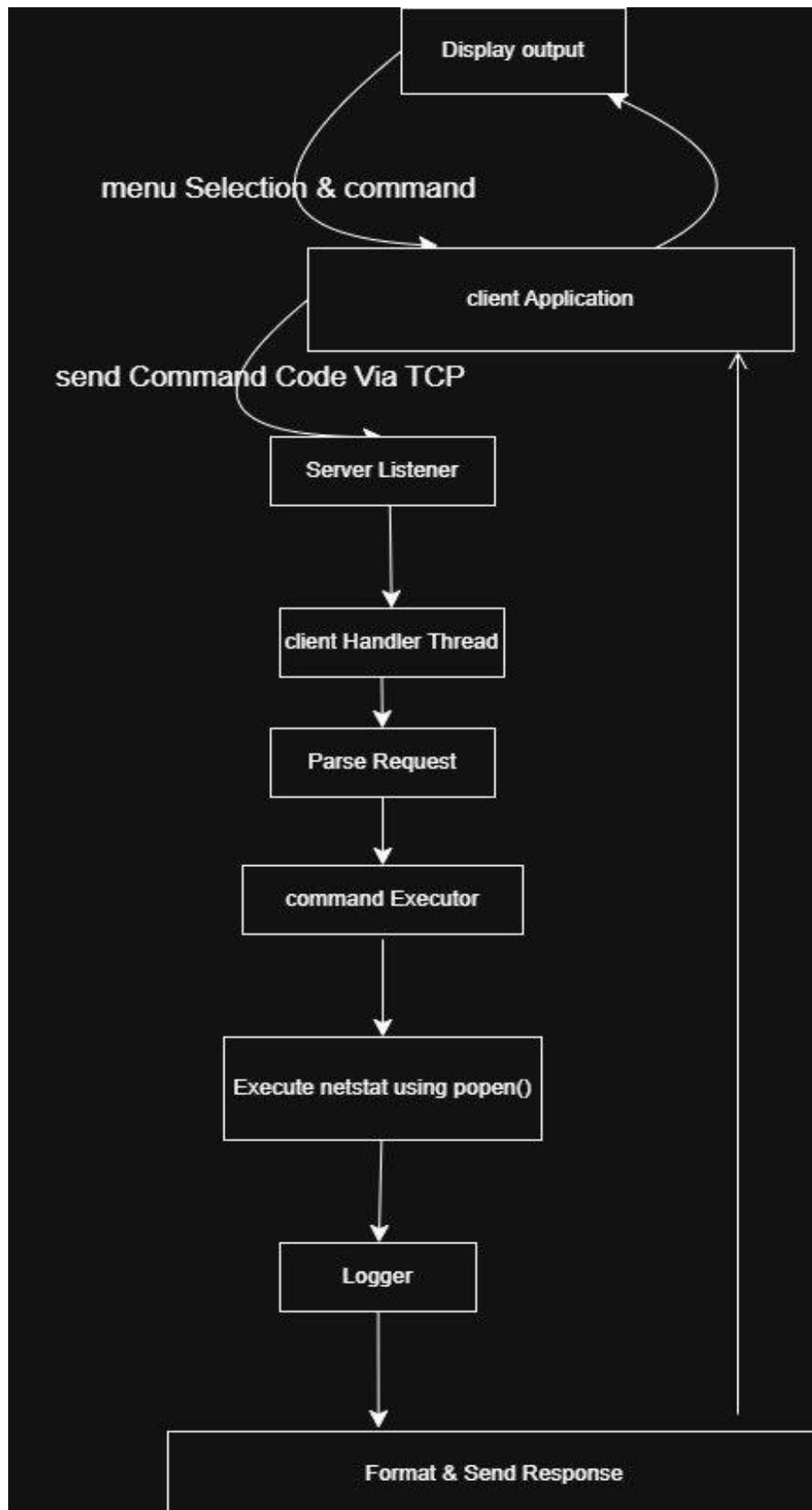
3.3.3. Activity Diagram server side request



3.3.4. Class Diagram



3.3.5 Data Flow Diagram



3.4 Files

The project mainly contains:

- Client source file
- Server source file
- Logging module
- Documentation files
- Test case files

The client sends requests and receives responses while the server processes Netstat commands.

3.5 User Interface

The project uses Command Line Interface (CLI).

Features:

- Menu-driven interaction
- Easy navigation
- Readable output display

3.6 Error Handling

Error handling is used to display proper error messages whenever an error occurs during client-server communication or while executing Netstat commands. This helps users understand the problem and identify where the execution failed.

- The system checks whether the client is properly connected to the server.
- Errors related to TCP socket creation and connection establishment are handled.
- Invalid menu options entered by the user are detected and proper messages are displayed.
- If the server fails to execute a Netstat command, an error message is returned to the client.
- If the client disconnects unexpectedly, the server properly closes the socket connection.

- Logging messages are generated for INFO, DEBUG, WARNING, and FATAL errors.
- If requested data is not available, the server sends an appropriate failure message to the client.

Example Error Messages

1. E_CONNECTIONFAILED – Displayed when the client fails to connect to the server.
2. E_SOCKETERROR – Displayed when socket creation fails.
3. E_INVALIDOPTION – Displayed when an invalid menu option is selected.
4. E_COMMANDFAILED – Displayed when execution of a Netstat command fails.
5. E_CLIENTDISCONNECT – Displayed when the client disconnects unexpectedly.
6. E_SERVERERROR – Displayed when the server encounters an internal processing error.
7. E_RESPONSENOTFOUND – Displayed when requested information is not available.

3.7 Help

Help will be provided through the project documentation prepared before implementation. The documentation explains the system architecture, workflow, modules, and usage of the application. Additional instructions for compiling, executing, and testing the project will also be included. The documentation will help developers, testers, and users understand the complete working of the system.

3.8 Performance

The performance of the system mainly depends on fast client-server communication and quick execution of Netstat commands. The server should efficiently process requests and return responses without unnecessary delay. The concurrent server design allows multiple clients to communicate with the server simultaneously while maintaining stable performance.

3.9 Security

The system uses TCP socket communication for reliable data transfer between the client and server. Only predefined Netstat commands are executed by the server to avoid unauthorized command execution. Invalid requests from the client are rejected by the server. Proper socket connection management is maintained to prevent resource leakage and unauthorized access. Logging mechanisms are also used to monitor system activities and detect failures or suspicious behavior.

3.10 Reliability

The Netstat Utility system is available whenever the server is running and listening for client requests. The server continuously accepts client connections through IP address and port number using TCP protocol. Since TCP provides reliable communication, the data sent between client and server is delivered correctly and in proper order. The system also supports multiple client connections simultaneously using multithreading.

3.11 Maintainability

The project follows a modular programming structure which makes the system easier to maintain and modify in the future. Different functionalities such as client handling, command processing, logging, and socket management are implemented as separate modules. This structure helps developers easily debug errors, update features, and improve the system without affecting the complete application.

3.12 Portability

The Netstat Utility system is designed to run on Linux operating systems that support TCP socket programming and Netstat utility. The application can be moved and executed on different Linux distributions with minimal changes. Since the project is implemented using standard C/C++ programming and Linux networking libraries, the code can be compiled and executed on multiple Linux environments using GCC/G++ compiler.

3.13 Reusability

The modules and components used in this project are designed to be reusable in future networking and client-server applications. Components such as socket handling, logging mechanism, command processing, and multithreading can be reused without major modifications. The modular programming structure also helps developers extend the system by adding new features and commands easily.

3.14 Application Compatibility

The Netstat Utility system uses TCP as its communication protocol between the client and server. The application is compatible with Linux systems supporting TCP/IP networking, socket programming, and Netstat utility. The client and server communicate through IP address and port number for exchanging requests and responses.

3.15 Resource Utilization

The system uses CPU resources for processing client requests and executing Netstat commands.

Memory resources are used for socket handling, storing command outputs, and managing multiple client threads. The resource utilization is minimal since the application mainly processes lightweight networking commands.

3.16 Major Classes and Characteristics

The major classes/modules used in the system are:

There are major classes/modules: Client, Server, Command Processor, and Logger System Modules

Low Level Design Document

4. Introduction

The aim of this document is to gather, analyze, and provide a detailed understanding of the complete Netstat Utility using Client-Server Architecture by defining the system design and implementation details. The intended audience includes all stakeholders involved in the system such as users, developers, testers, client, and server administrators. The detailed low-level design of the Netstat Utility system is provided in this document.

4.1 Purpose

- The Netstat Utility is a Linux-based client-server application used to remotely monitor network-related information from a server system.
- The project is implemented using TCP socket programming in C/C++ language and uses TCP protocol for reliable communication.
- The system allows clients to request different Netstat operations such as protocol statistics, routing table information, TCP/UDP connections, and LISTEN or ESTABLISHED network states.
- The server processes the requested Netstat commands and sends the generated output back to the client.
- The system supports multiple clients simultaneously using multithreading.
- Any client can connect to the server if it knows the server IP address and port number.

4.2 Document Conventions

The document uses the “Times New Roman” font with bold style for headings. The main heading size is 18 points. The sub-heading size is 14 points and the content under each heading is of 12 points size. All spacing used in the document follows the normal/default spacing format of MS Word.

4.3 Intended Audience and Reading Suggestions

This project is intended for users who want to remotely monitor and view networking information from a Linux system. This document is intended to be read by developers, testers, project managers, reviewers, and users. The document contains technical terms related to networking, Linux systems, socket programming, and client-server communication, which should be understood by the readers.

4.4 References

The references are:

- Linux Socket Programming Documentation
- Linux Netstat Utility Documentation
- TCP/IP Networking Concepts
- <https://www.geeksforgeeks.org/socket-programming-cc/>
- <https://man7.org/linux/man-pages/man8/netstat.8.html>

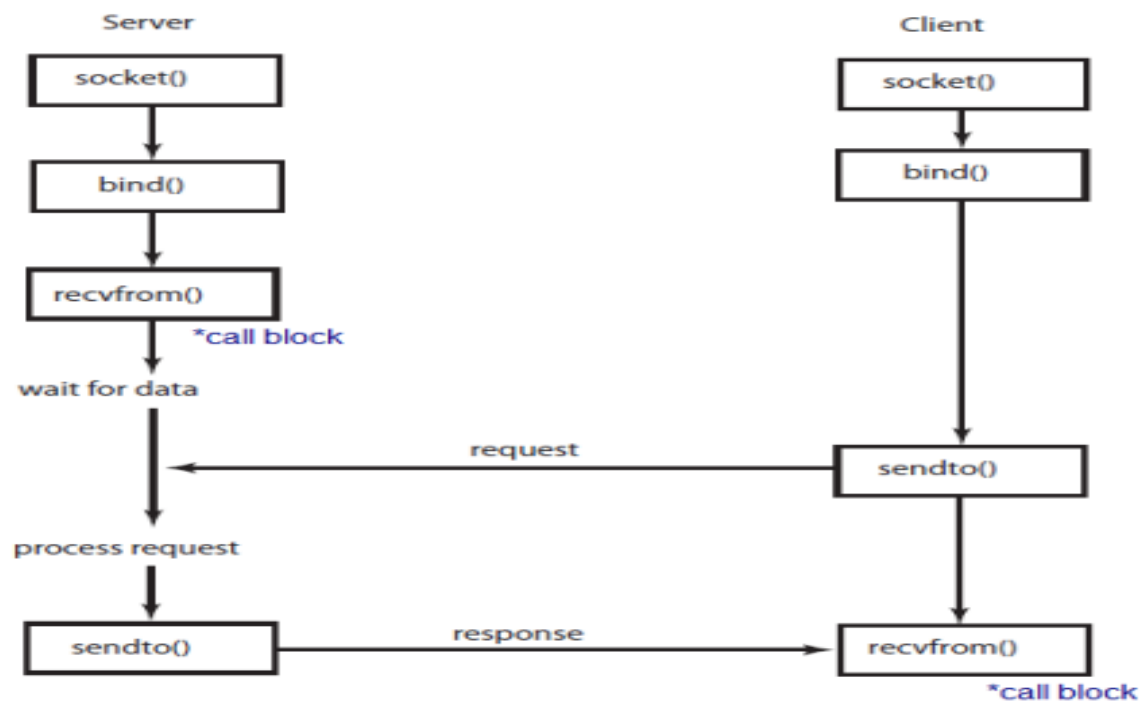
5. Detailed System Design

The system design is as follows:

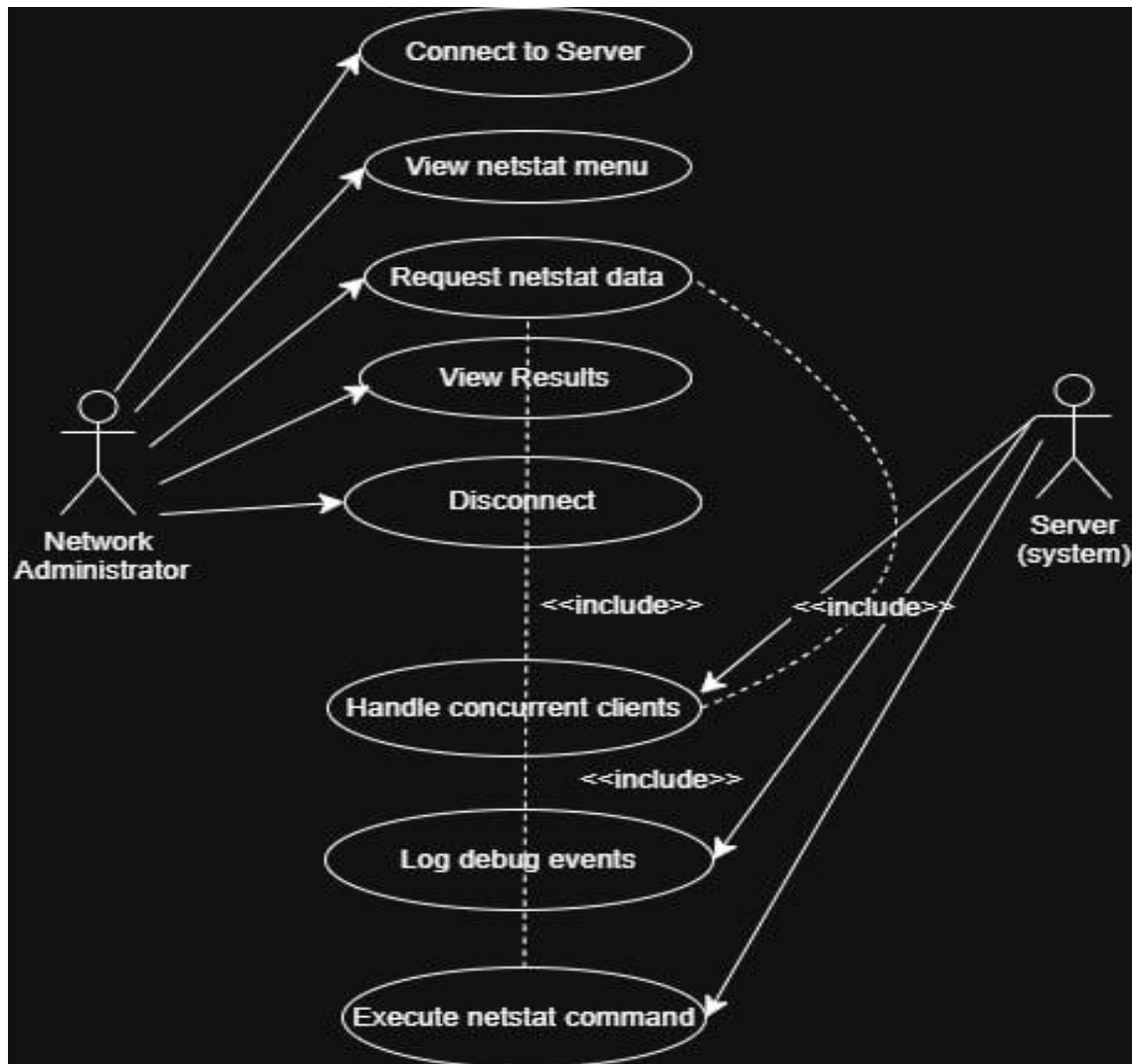
5.1 Design Descriptions

The main design features include client-server architecture, command processing, socket communication, logging mechanism, and concurrent client handling. The design also includes user interface, process flow, and error handling. In order to make these designs easier to understand, the design has been illustrated using diagrams such as Use Case Diagram, Class Diagram, and Data Flow Diagram.

5.2 Connection Establishment between Client and Server



5.3. Use Case Diagram



5.4 Design and Implementation Constraints

- The system is built using C/C++ programming language.
- Linux environment is mandatory.
- TCP protocol must be used.
- Netstat utility should be available in the Linux system.

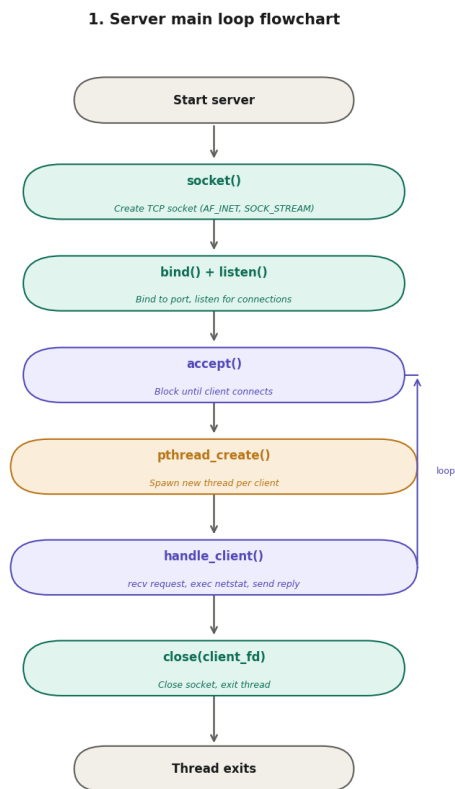
5.5 User Interface

- Desktop or Linux machine with network connectivity.
- Command Line Interface (CLI).
- Menu-driven interaction between user and system.

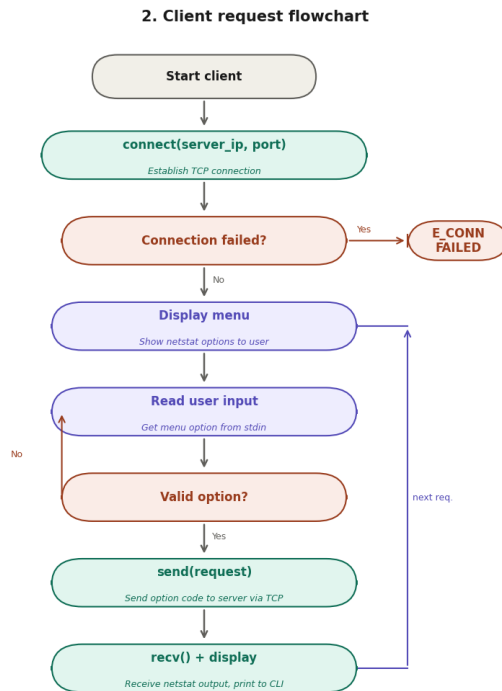
5.6 Security

Since the system uses client-server communication, proper socket management and request validation mechanisms are maintained to avoid unauthorized access and invalid command execution. Only predefined Netstat commands are allowed to be executed by the server. Logging mechanisms are also used to monitor client connections and system activities.

5.7 Server Main Loop Flowchart

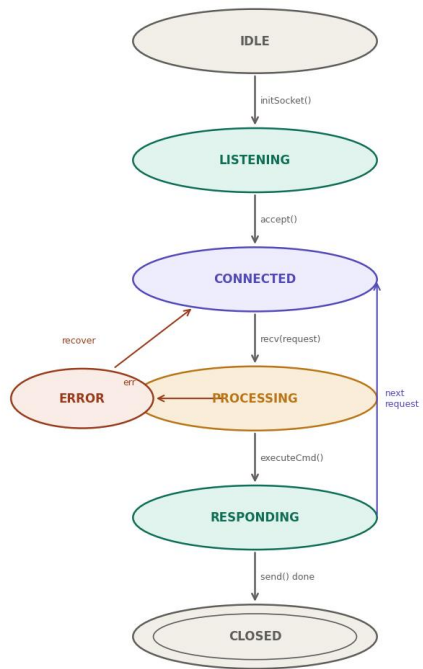


5.8 Client Request Flowchart



5.9 State Diagram

4. Server connection state diagram



5.11 Thread Interaction Diagram

5. Thread interaction diagram

